

SMART INDIA HACKATHON 2026



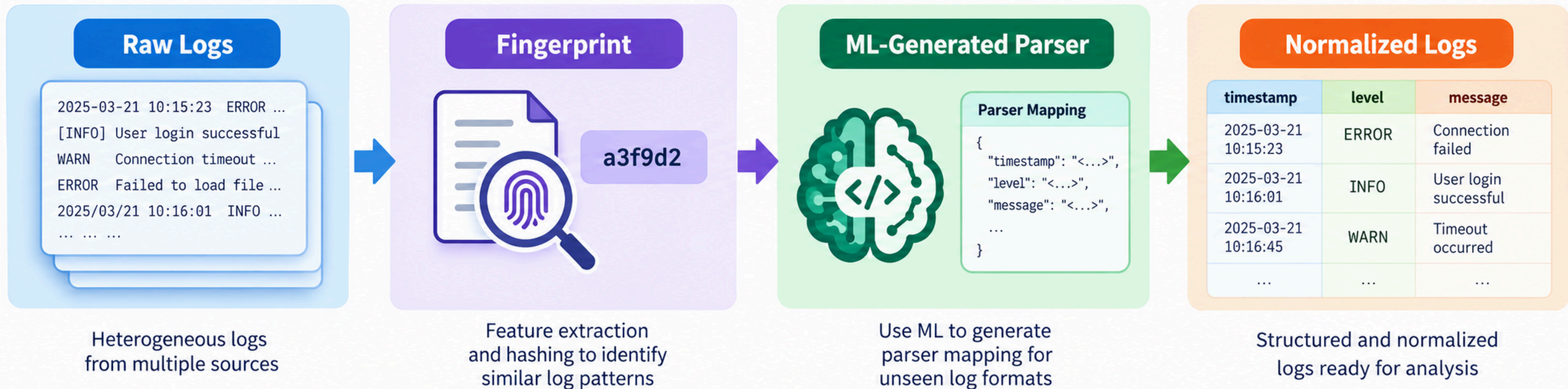
- **Problem Statement ID** – SIH26156
- **Problem Statement Title**- Universal Log Pre-processing Framework
- **Theme**- Blockchain & Cybersecurity
- **PS Category**- Software
- **Team ID**-
- **Team Name**- EleventhHour



Intelligent Log Normalization & Parser Generation System

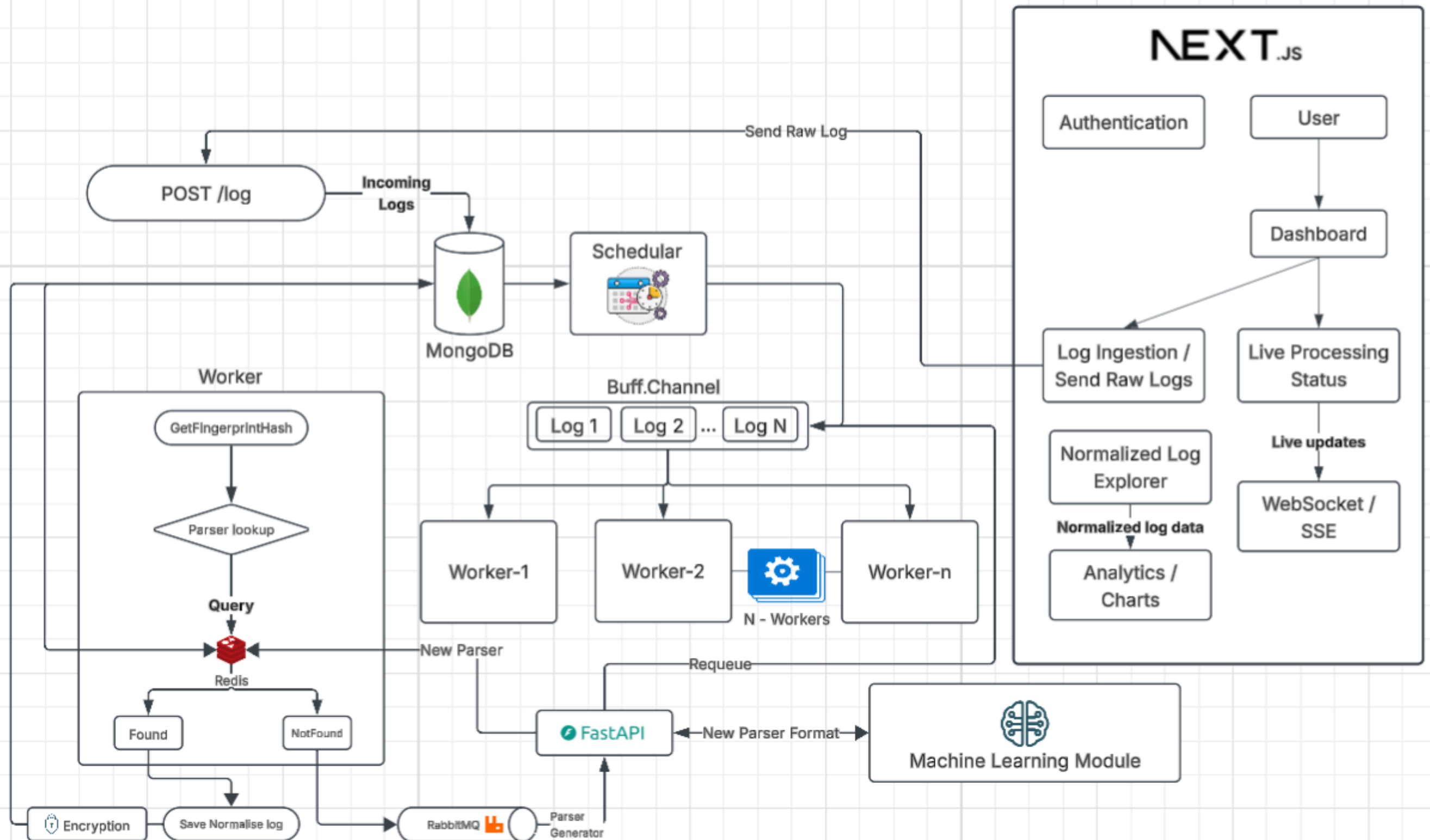
Zero-Manual-Regex Ingestion Engine

From **Raw Logs** → **Fingerprint** → **ML-Generated Parser** → **Normalized Logs**



Technology Used

- **Backend Engine:** Go
- **Message Broker:** RabbitMQ
- **High-Speed Cache:** Redis
- **ML / Parser Synthesizer:** Python + FastAPI + Pydantic + Groq API
- **Database & Persistence:** MongoDB (Event store & audit log)
- **Frontend UI:** Next.js Tailwind



Technical Feasibility

- Uses proven technologies: Database, Redis, RabbitMQ and ML services
- 120+ parallel workers enable concurrent log processing.
- Fingerprinting reduces repeated parser-generation requests.
- Redis caching provides fast parser retrieval.
- RabbitMQ provides reliable asynchronous processing.

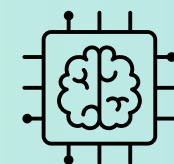
Scalability

- Workers can be scaled horizontally as log volume increases.
- Queue-based architecture prevents processing bottlenecks.
- Parser mappings are reusable across similar logs.
- Asynchronous ML requests prevent the main pipeline from blocking.

Viability

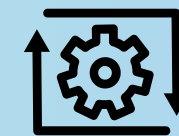
- Existing parsers are reused instead of regenerated.
- ML is invoked only for previously unseen fingerprints.
- Reduces manual parser creation and maintenance.
- Architecture can support increasing log sources and formats.

Future Scope



Fine Tuning

Fine-tune the LLM to generate more accurate and reliable parsers for previously unseen log formats.



Automatic Logs Collection

Automatically collect logs from the user's system through an installed agent, eliminating the need for manual log uploads.

Transforming Raw Logs into Actionable Data



Automated Parsing

Minimizes manual parser development



High Throughput

~120 workers process logs concurrently



Self-Learning

ML generates mappings for unseen log formats



Faster Processing

Redis enables rapid parser retrieval



Parser Reusability

One learned mapping serves similar logs



Structured Data

Normalized logs simplify downstream analysis



Reliable Pipeline

Queues provide asynchronous, controlled processing



Scalable Architecture

Can handle increasing log volumes and sources



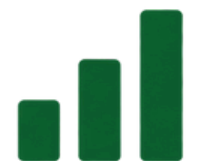
Improved Decision Making



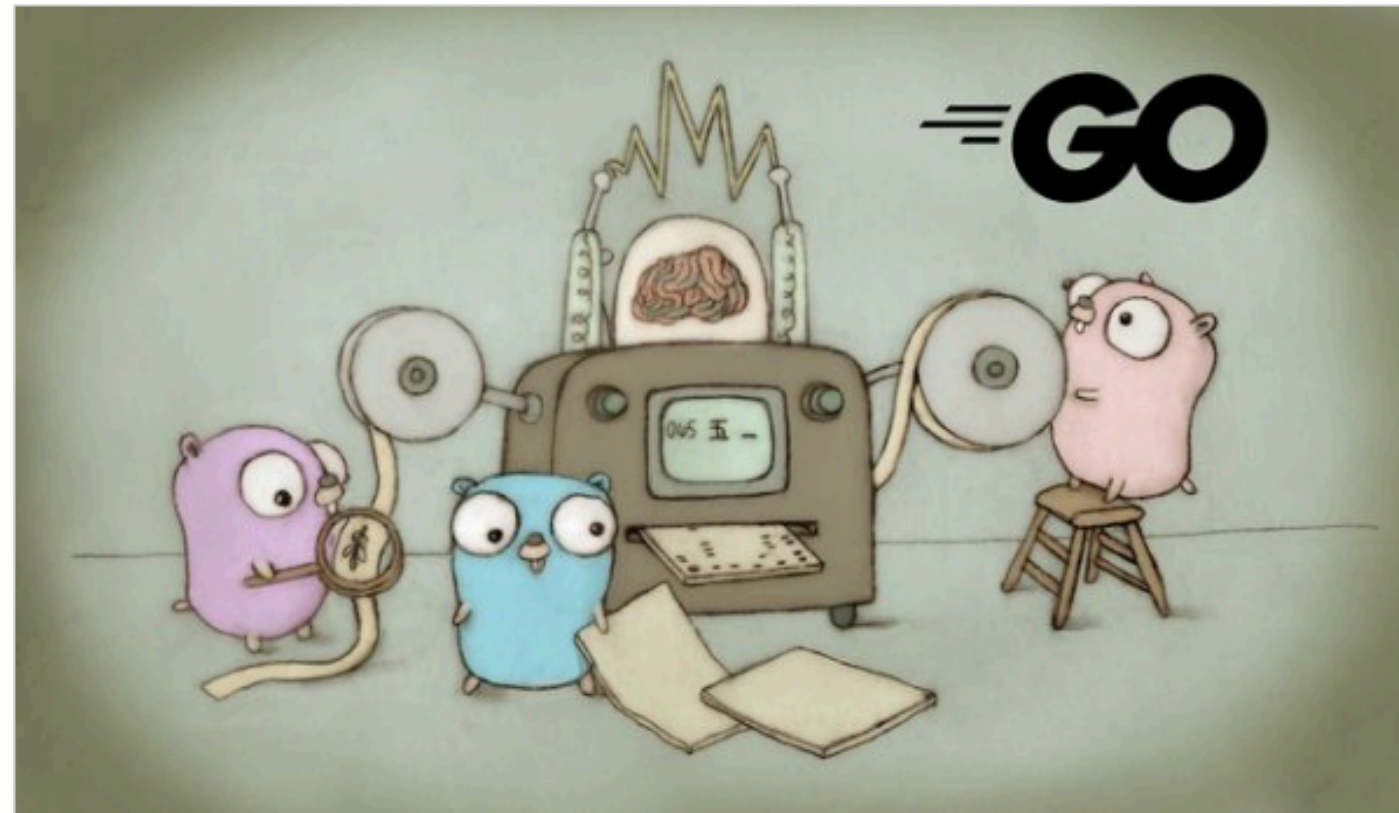
Faster Monitoring



Reduced Manual Effort



Better Analytics



Go Concurrency Patterns: Pipelines and cancellation - The Go Programming Language

How to use Go's concurrency to build data-processing pipelines.

 golang

Master the Go Worker Pool Pattern

Build efficient concurrent systems with a Go worker pool. See how to manage goroutines and channels for maximum throughput.

Go FAQ / Apr 25



Redis cache-aside

Cache database reads in Redis with TTL-bounded staleness.

 Docs